In defense of *Some applications of formal mathematics*

John V Siratt

University of Notre Dame Department of Mathematics
NASA Langley Research Center Formal Methods Research Group

June 24, 2024

Opening comments

A (not so) hypothetical

- ⋆ Your company has developed an autonomous vehicle
  - **self-driving car**
  - package delivery drone
  - robotic ice cream truck?
- ⋆ You are part of the safety team
  - **recognize traffic signals**
  - release package only at ground level
  - not mix cleaning solutions into the rocky road

Formal methods are "... mathematically rigorous techniques and tools for the specification, design and verification of software and hardware systems."

– NASA Langley Formal Methods Research Program website

Proof assistants are one tool in formal methods.

Verification

Your task:
Verify an algorithm P_c that determines if a traffic light is red works correctly ($\psi(c)$).

$$R = \{i \mid \psi(i)\}$$

Determine if $c \in R$.

Rice's theorem – only computable index sets are $\emptyset$ and ω.

Rice-Shapiro theorem – essentially, r.e. index sets are those for which we can determine if an algorithm belongs by only checking it's behavior at a finite number of points.

So interesting properties always define "difficult" index sets. Job security!

Index set complexity

There are degrees of uncomputability. The halting problem is undecidable but semi-decidable (it's an r.e. set).

In general, index sets can be arbitrarily complex.

The smallest nonempty index sets (which we will call *minimal*) are those of the form $I_f = \{i \mid P_i \textit{ computes } f\}$ for some fixed computable function f.

We can bound the complexity of minimal index sets.

Result

- Suppose f has an infinite computable domain (including if f is a total function). Then $I_f \equiv_1 Tot = \{i \mid \forall n,\ P_i(n)\downarrow\}$. Hence I_f is Π^0_2-complete.
- The complement of $I_0 = \{i \mid P_i \textit{ is nowhere defined}\}$ is equivalent to the halting set, so it is Π^0_1-complete.
- If f has a nonempty domain, then $\overline{I_0} \leq_1 I_f$.
- If f has a finite domain, then $I_0 \leq_1 I_f$ and $\overline{I_0} \leq_1 I_f$. So I_f is strictly Δ^0_2.
- If f, g both have finite domains, then $I_f \equiv_1 I_g$.
- $\emptyset' \leq_T I_f \leq_T \emptyset''$ – computing a minimal index requires at least a halting oracle, but never more than a termination oracle.

(*Note*: halting asks if a program stops on a particular input, termination asks if a program stops on **all** inputs. More on termination to come.)

Partial solutions

We don't need the solution for every case – just enough to be useful.

What is "good enough"?

- no more complex than r.e.
- obvious variations, so probably infinite
- two equally good (disjoint) solutions combine to make a better solution?

(not every set has an infinite r.e. subset...)

Maybe "good enough" if we can only improve it by adding a special case here or there?

- Let X be an r.e. subset of a non-r.e. set Y. We say that X is *maximal* in Y when every r.e. set Z for which $X \subset Z \subset Y$ is such that $X \equiv_{fin} Z$.

Result

Unfortunately, this does not work well for index sets.

- Any index set (indeed, any set which is a *cylinder*) has no maximal r.e. subset.

The intuition here is that since every index set is a cylinder, it can be decomposed into an infinite disjoint union of r.e. subsets. No matter how many of these we combine, there are always infinitely many new ones we could add to a partial solution.

Termination analysis

Perhaps we need to look at a specific problem.

We've seen many verification problems reduce to Tot.

Termination analysis – is P_c defined for every input?
That is, is $c \in Tot$.

(We probably don't want our car to bluescreen as we approach a stop light...)

One method of termination analysis

Matrix-weighted graph termination

- Every recursive call is represented by a vertex labeled with a *calling context*
- These consist of the *condition* under which that recursion is performed, and the *transformation* we use to compute the parameters we pass into the recursion
- If the program can transition from one recursive call to another, there must be a directed edge indicating this (it's ok to have extra edges)
- *n measure functions*, $\mu_1, \ldots, \mu_n$
- The graph edges are weighted with $n \times n$ matrices that record information on whether the parameters are decreasing by some combination of measure functions

Soundness conditions

Let $a = (A, f)$ and $b = (B, g)$ be calling contexts

Edge soundness – If there is not an edge $a \to b$, then

$$\forall x, A(x) \implies \neg B(f(x))$$

Measure soundness – For every i, j,

$$M_a(i, j) = \star \to \forall x, A(x) \to \mu_i(x) > \mu_j(f(x))$$

$$M_a(i, j) = 1 \to \forall x, A(x) \to \mu_i(x) \geq \mu_j(f(x))$$

MWG Termination: An algorithm P terminates if and only if there exists an edge sound and measure sound MWG for P for which all circuits are positive – that is, if the matrix product of the circuit has $\star$ on the diagonal.

Result

We have to simplify these issues some if we want to find a partial decision procedure.

Some strong simplifications:

- single calling context
- condition must be primitive recursive
- transformation must be identity
- a single measure function – identity
- if an edge exists, it can only be labeled with $[\star]$

Result. Even under such strong restrictions, we can embed any Π^0_1 set into each soundness problem.

However...

When was the last time you saw an algorithm like this?

$$f(n,m) = \begin{cases} f(n^{n+m-1}, m-1) & \prod_{i=0}^{n} m^{n^i} < m^{\sum_{i=1}^{n} i} \\ \dots & \dots \end{cases}$$

Practical algorithms don't often need full primitive recursiveness for their calling contexts.

Machine learning

Your task (amended): P_c is a neural net

Identifying red lights is a *binary classification* problem.

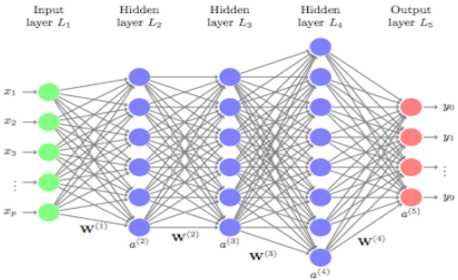
Any neural network can be used for binary classificcation:

- add a neuron at the end with a threshold activation

$$step(r) = \begin{cases} 1 & r > 0 \\ 0 & o.w. \end{cases}$$

Feed forward neural nets

Deep learning often uses feed forward neural nets (ffnn)



$$F(W \cdot v + B)$$

A common activation function is the rectified linear unit

$$ReLU(r) = \begin{cases} r & r > 0 \\ 0 & o.w. \end{cases}$$

Things we know

- the functions computed by ReLU-activated ffnns are precisely the (positive) continuous piecewise linear functions (Arora, et al, 2018)
- the decision boundary of a two-layer ReLU-activated ffnn can be described by a step-activated ffnn (Pan and Srikumar 2016)
- various related results

Result

Using an approach similar to Pan and Srikumar, we can extend their result to networks of arbitrary size.

- Every ReLU-activated binary classifier can be translated into a statement in an unquantified fragment of first order arithemetic

$$\psi(w_1 \cdot v + b_1 > 0, \dots, w_k \cdot v + b_k > 0)$$

- This is a propositional formula and a single layer of step-activated neurons
- The other direction can be shown for a full characterization, but we are primarily interested in a constructive proof that provides a translation algorithm

(Ongoing research... formalization... bite-sized proofs...)

Proof complexity

Your task: Get to proving!

But formal proofs can be really long...

Proof theory

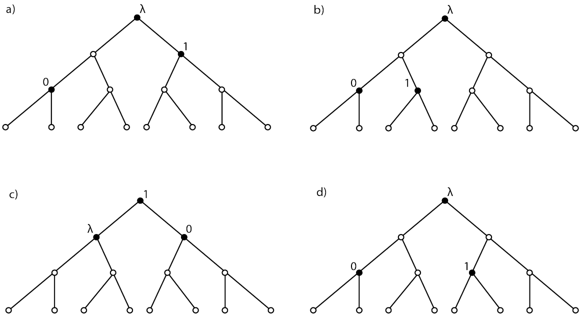
- There exists a proof system for propositional logic which has "short" (i.e., polynomial) proofs of all tautologies if and only if $P = NP$ (Cook 1971)
- In RCA_0, we only obtain polynomial decrease in size for proofs of $\forall\Sigma^0_2$ statements by assuming WKL and RT^2_2 (Kolodziejczyk, Wong, and Yokoyama 2020)

Trees and strong subtrees

The latter result used proof-theoretic forcing to extend a conservation result based on *largeness*.

Can we do this for Milliken's (or another tree theorem)?

Need *strong subtrees*: (binary trees for technical reasons)



Largness

... has been used for proof theoretic results. Earliest example probably the Paris-Harrington result, extended by others such as Ketonen and Solovay.

Let $X \subset_{fin} \omega$.

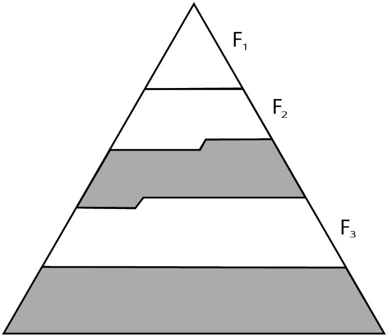
- X is ω-large if $|X| \geq 1 + \min X$ (Paris-Harrington largeness essentially)
- X is $[\omega^n \cdot k]$-large if $X = Y_1 \sqcup \cdots \sqcup Y_k$ with each Y_i being ω^n-large
- X is ω^{n+1}-large if $X = \{\min X\} \sqcup Y$ with Y being $[\omega^n \cdot \min X]$-large
- X is α-large if it contains (or contains a subset dominated by) an α-large set

For trees we need to preserve structure.

Slices, stacks, level sets

A *slice* out of a labeling.

An (ω^n, ω^m)-*stack*: ω^n-slices, largest labels in each form an ω^m-large set.



A *level set* contains the largest labels for each level in some portion of the tree.

Results

Theorem. An ω^{n+2}-slice S with $\min S \geq 2$ has an ω^n-large level set. (Actually $[\omega^n \cdot (\min S - 1)]$-large.)

Theorem. An ω^{n+2m}-slice S with $\min S \geq 2$ contains an (ω^n, ω^m)-stack.

A tree largeness result. (An ordinal-valued Ramsey number.)

Theorem. Let T be an ω^{3n+9}-labeling with $\min T \geq 2$. For any 2-coloring of the vertices of T there exists an ω^n-large set labeling a monochromatic strong subtree of T.

The best results I got for Milliken's for binary trees were above ω^ω, which is too large to use the method.

How many strong subtrees?

Let $S(n, m)$ be the number of strong subtrees of height m in the binary tree of height n.

Finite Milliken's – For every tree of height N and every coloring of it's strong subtrees of height k, there exists a monochromatic strong subtree of height m.

- number of possible colorings is related to $S(N, k)$
- $S(N, m)$ possible solutions
- solutions would have $S(m, k)$ subtrees of the same color

The number of subtrees seemed important but I couldn't find it in the literature

... So I counted

Counting

It got out of hand pretty quickly, which fed my curiosity

$S(n,m)$	$n=1$	2	3	4	5	6	7
$m=1$	1	3	7	15	31	63	127
2	0	1	7	35	155	651	2667
3	0	0	1	23	403	6603	106299
4	0	0	0	1	279	71827	18394315
5	0	0	0	0	1	65815	4313323667
6	0	0	0	0	0	1	4295033111

$$S(8,7) = 18446744078004584727$$

Result

General result gives elementary expressions for diagonals.

$$S(m+1,m) = 3 + \sum_{j=1}^{m-1} 2^{2^j}$$

$$S(m+2,m) = 7 + 3\left(\sum_{j=1}^{m-1} 2^{2^j}\right) + \sum_{1\leq i,j\leq m-1} 2^{2^i+2^j}$$

$$S(m+3,m) = 15 + 7\left(\sum_{j=1}^{m-1} 2^{2^j}\right) + 3\left(\sum_{1\leq i,j\leq m-1} 2^{2^i+2^j}\right)$$
$$+ \sum_{1\leq i,j,k\leq m-1} 2^{2^i+2^j+2^k}, \; \textit{and so on}\ldots$$

Quick summary

What a long, strange trip it's been...

2012 started working at COTO, employee tuition waiver

2015 first emailed Peter about studying logic at Notre Dame

2016 introduced to Langley Formal Methods team

2017 first internship with Langley team, started MS program

2019 finished MS, came to Notre Dame to study mathematical logic with intent to apply it to FM

2021 hired by Langley as part of Pathways program

2024 PhD?

Questions or comments?